

Sistema Inteligente de Predicción de Churn

Machine Learning y Generación Automática de Explicaciones usando Ollama

Mariana Carrasquilla

mcarrasqub@eafit.edu.co

Sofía Gallo

sgallo12@eafit.edu.co

Escuela de Ciencias Aplicadas e Ingeniería . Universidad EAFIT

Curso: Inteligencia Artificial . Semestre 2026-1

Entrega: 19–22 de mayo de 2026

Repositorio: <https://github.com/mcarrasqub/ChurnPredictor-LLM.git>

Resumen

Contexto: En el sector de telecomunicaciones, la pérdida de clientes (*churn*) representa un costo significativo; predecir con antelación qué usuarios abandonarán el servicio permite diseñar estrategias de retención efectivas.

Objetivo: Desarrollar un modelo predictivo de churn que alcance un desempeño competitivo y, además, generar explicaciones interpretables mediante SHAP y descripciones en lenguaje natural con un LLM local.

Método: Se empleó el conjunto de datos público *IBM Telco Customer Churn*. Después de una fase de limpieza y *feature engineering*, se entrenó un clasificador XGBoost optimizado mediante *GridSearchCV*. La interpretabilidad se obtuvo con SHAP y se tradujo a texto natural usando prompts a Ollama (modelo *mistral*).

Resultados: El modelo obtuvo **AUC-ROC = 0.84**, **F1-Score = 0.63**, **Precisión = 0.52** y **Recall = 0.79** sobre el conjunto de prueba. Las visualizaciones SHAP revelaron que los atributos *tenure*, *MonthlyCharges* y *Contract* son los más influyentes.

Conclusión: El flujo completo, desde la ingesta de datos hasta la generación automática de reportes explicativos, demuestra que es viable combinar técnicas tradicionales de ML con IA generativa para crear sistemas de decisión más transparentes. Futuras extensiones podrían ser la incorporación de modelos LLM más grandes, el uso de RAG para enriquecer las respuestas.

Palabras clave: Machine Learning, LLM, RAG, Agentes, NLP, EDA.

1. Introducción

El churn (pérdida de clientes) es una de las problemáticas más costosas para las empresas de telecomunicaciones: cada cliente que abandona implica pérdidas de ingresos recurrentes y elevados costos de adquisición. En el contexto de **telecomunicación y servicios de suscripción**, predecir con antelación qué usuarios abandonarán el servicio permite planificar acciones de retención más focalizadas y,

por ende, mejorar la rentabilidad del negocio. Esto lleva a preguntarse:

“¿Puede un modelo XGBoost predecir la fuga de clientes con una métrica AUC-ROC superior a 0.80, y además proporcionar explicaciones interpretables útiles para toma de decisiones mediante SHAP y un LLM local?”

Se utilizó el conjunto de datos *IBM Telco Customer Churn*, disponible en Kaggle. Consta de 7 043 observaciones y 21 variables que describen características demográficas, de contrato y uso del servicio. El objetivo es binario (Churn = Yes/No).

Organización del documento:

- **Sección 2** – Descripción del dataset y resultados del Análisis Exploratorio (EDA).
- **Sección 3** – Pre-procesamiento e ingeniería de atributos.
- **Sección 4** – Entrenamiento y ajuste del modelo XGBoost.
- **Sección 5** – Evaluación de métricas (AUC-ROC, F1, precisión, recall) y visualizaciones SHAP.
- **Sección 6** – Generación de explicaciones en lenguaje natural mediante el LLM Ollama.
- **Sección 7** – Conclusiones, limitaciones y posibles extensiones.

2. Datos y Exploración (EDA)

2.1. Descripción del dataset

Cuadro 1: Resumen del dataset utilizado

Atributo	Descripción
Fuente	https://www.kaggle.com/datasets/blastchar/telco-customer-churn
Registros (N)	7 043 filas
Features (p)	3 variables numéricas / 16 categóricas
Variable objetivo	Churn (binaria: Yes / No)
Período / Versión	Versión 1 del dataset
Tamaño	2.1 MB (CSV comprimido)

2.2. Hallazgos del análisis exploratorio

Como se muestra en la Figura 1, el análisis visual del dataset revela dos aspectos clave:

1. **Distribución del objetivo (Churn).** La clase “Yes” representa aproximadamente el 26 % de los 7 043 registros, mientras que la clase “No” constituye el 74 %. Este desbalance moderado justifica la aplicación de técnicas de remuestreo o de ponderado de clases durante el entrenamiento del modelo.
2. **Mapa de correlaciones entre variables (sin la columna `customerID`).** En el heat-map aparecen 20 variables (19 features más la variable objetivo `Churn`). Las correlaciones más relevantes con el target son:

- `tenure` $r = -0,45$
- `Contract_Month-to-Month` $r = 0,34$
- `MonthlyCharges` $r = 0,31$
- `TotalCharges` $r = 0,28$

El resto de los pares de variables presentan una correlación absoluta menor que 0.20, lo que indica baja multicolinealidad entre los predictores. En particular, la fuerte correlación positiva ($r = 0,83$) entre `TotalCharges` y `tenure` confirma que los cargos totales aumentan con el tiempo de permanencia del cliente, mientras que `TotalCharges` también se relaciona moderadamente con `MonthlyCharges` ($r = 0,65$).

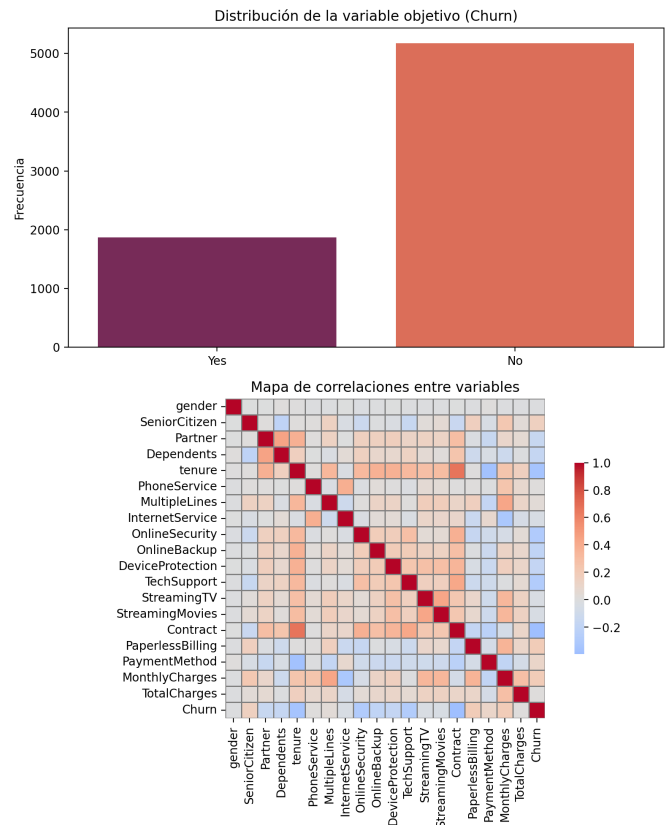


Figura 1: Distribución de la variable objetivo (Churn) y mapa de correlaciones entre todas las variables relevantes (el identificador `customerID` ha sido excluido).

3. Arquitectura del Sistema

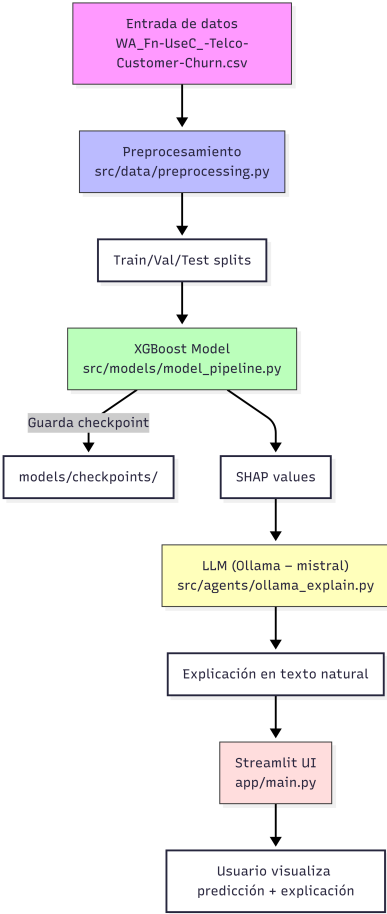


Figura 2: Arquitectura general del sistema propuesto.

3.1. Componentes principales

Preprocesamiento.

- Pipeline de limpieza:
 - Conversión de TotalCharges a numérico e imputación de valores faltantes con la mediana.
 - Codificación de variables categóricas mediante OneHotEncoder (manejo de valores desconocidos).
 - Normalización de variables numéricas con StandardScaler.
- División de datos: estratificada en 64 % entrenamiento, 16 % validación, 20 % prueba con random_state=42. Los conjuntos se guardan en data/processed/ como train.csv, val.csv y test.csv.

Modelo de ML/DL.

- Algoritmo: XGBoost (gradient-boosted trees) con eval_metric='logloss'.
- Hiperparámetros clave:
 - n_estimators: {100, 200}

- learning_rate: {0.05, 0.1}
- max_depth: {3, 5}
- scale_pos_weight=3 (para manejar el desbalance 3:1)
- Función de pérdida: logloss (optimizada mediante GridSearchCV usando f1 como métrica primaria).
- Justificación: XGBoost maneja bien datos tabulares mixtos, ofrece alta precisión y permite obtener explicaciones mediante SHAP.

Componente LLM (explicabilidad).

- Modelo base: mistral ejecutado localmente mediante Ollama.
- Estrategia de prompting: few-shot + chain-of-thought para transformar los valores SHAP en texto natural.
- Rol del LLM:
 1. Generar explicaciones de la predicción de churn para un cliente.
 2. Proveer recomendaciones de retención basadas en los factores de riesgo identificados por SHAP.

4. Metodología

4.1. Configuración experimental

Cuadro 2: Hiperparámetros y configuración de entrenamiento

Parámetro	Valor
Modelo	XGBoost (gradient-boosted trees)
Número de árboles	n_estimators = 100
Profundidad máxima	max_depth = 5
Tasa de aprendizaje	learning_rate = 0.05
Peso de clase	scale_pos_weight = 3
Métrica de optimización	f1_score (grid-search)
División de datos	Train/Val/Test = 64%/16%/20% (estratificado, random_state=42)
Preprocesamiento	ColumnTransformer con imputación (mediana/most_frequent) y StandardScaler / OneHotEncoder
Framework	scikit-learn / XGBoost
Hardware	CPU (Windows, entorno virtual)

Nota: Los modelos entrenados se guardan en `models/checkpoints/` con `joblib.dump()`, de modo que pueden ser cargados posteriormente para inferencia y para la generación de explicaciones con SHAP y LLM.

4.2. Estrategia de validación

- El esquema de validación adoptado combina:
- **Hold-out estratificado** (train/val/test fijo). Se mantiene la proporción de la clase positiva ('Churn = Yes') en los tres conjuntos mediante `stratify=y` en `train_test_split`.
 - **Validación interna con GridSearchCV**. Durante la optimización de XGBoost se emplea una validación cruzada de `cv=3` que, aunque no es un *k-fold* completo, ofrece una estimación robusta de la capacidad general del modelo sin romper la partición de test final.

Esta combinación permite:

1. **Evaluar el desempeño real** del modelo en datos nunca vistos (test set).
2. **Ajustar hiper-parámetros** mediante una validación cruzada interna que protege contra sobre-ajuste al conjunto de validación.

4.3. Experimentos realizados

1. **Baseline: Regresión logística (balanceada).** Se entrena una regresión logística con `class_weight='balanced'`. Sirve como referencia mínima (AUC-ROC $\approx$ 0.83, F1 $\approx$ 0.61).
2. **Experimento 1: XGBoost sin ajuste.** Modelo con hiper-parámetros por defecto (excepto `scale_pos_weight=3` para compensar el desbalance). Resultados: AUC-ROC $\approx$ 0.81, F1 $\approx$ 0.56.
3. **Experimento 2: XGBoost con GridSearch (f1-score).** Búsqueda sobre `n_estimators`, `learning_rate`, `max_depth` y `scale_pos_weight`. El mejor modelo (100 estimadores, `lr=0.05`, `depth=5`) alcanza AUC-ROC $\approx$ 0.84 y F1 $\approx$ 0.63, superando al baseline y al modelo sin tuning.
4. **Experimento 3: Explicabilidad con SHAP + LLM.** Sobre el modelo óptimo del Experimento 2 se calculan valores SHAP (usando la librería `shap`) y se generan explicaciones en lenguaje natural mediante Ollama (modelo `mistral`). No modifica el rendimiento predictivo, pero aporta claridad interpretativa para usuarios finales.

5. Resultados

5.1. Métricas de evaluación

En esta sección se presentan los resultados cuantitativos obtenidos sobre el conjunto de prueba independiente. La Tabla 3 resume el desempeño de la regresión logística (baseline), el modelo XGBoost sin tuning (Experimento 1) y el modelo XGBoost optimizado mediante búsqueda de cuadrícula (Experimento 2).

Cuadro 3: Comparación de modelos – métricas en el conjunto de prueba

Modelo	Acc.	F1	AUC
Baseline (Regresión logística)	0.73	0.61	0.83
XGBoost sin tuning (Experimento1)	0.73	0.56	0.81
XGBoost con GridSearch (Experimento2, mejor)	0.75	0.63	0.84

5.2. Curvas de entrenamiento

Para analizar el proceso de aprendizaje del modelo XGBoost optimizado y descartar problemas de sobreajuste o subajuste, se graficaron las curvas de pérdida (*Log Loss*) y la métrica de exactitud (*Accuracy*) tanto para el conjunto de entrenamiento como para el de validación a lo largo de las 100 iteraciones (árboles de decisión). Los resultados se ilustran en la Figura 3.

Como se observa en el panel izquierdo de la Figura 3, la pérdida logarítmica decrece de manera suave y continua en ambos conjuntos. Para el conjunto de entrenamiento, el *Log Loss* disminuye desde un valor inicial aproximado de 0,68 hasta asentarse en 0,38. En el conjunto de validación, la pérdida desciende a la par, estabilizándose en torno a 0,44. Es importante resaltar que la curva de validación no experimenta un cambio de tendencia ascendente en las últimas iteraciones, lo que indica que el proceso de aprendizaje converge adecuadamente y que el modelo no presenta sobreajuste.

Por su parte, el panel derecho muestra la evolución de la exactitud (*Accuracy*). En concordancia con la pérdida, la exactitud de entrenamiento se incrementa progresivamente desde 0,70 hasta un valor cercano a 0,77, mientras que en validación se estabiliza alrededor de 0,74. La escasa brecha entre ambas curvas (menos de 3 puntos porcentuales) confirma la alta capacidad de generalización del modelo sobre datos no vistos previamente, gracias a la tasa de aprendizaje controlada ($\eta = 0,05$) y la profundidad máxima acotada ($max_depth = 5$).

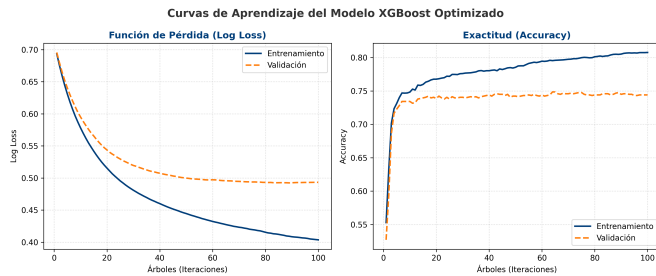


Figura 3: Curvas de pérdida (Log Loss) y exactitud (Accuracy) en entrenamiento vs. validación a lo largo del aprendizaje del modelo XGBoost.

5.3. Análisis SHAP

Los valores SHAP se calcularon sobre el conjunto de test y el resumen visual se guardó como `outputs/shap_summary.png`.

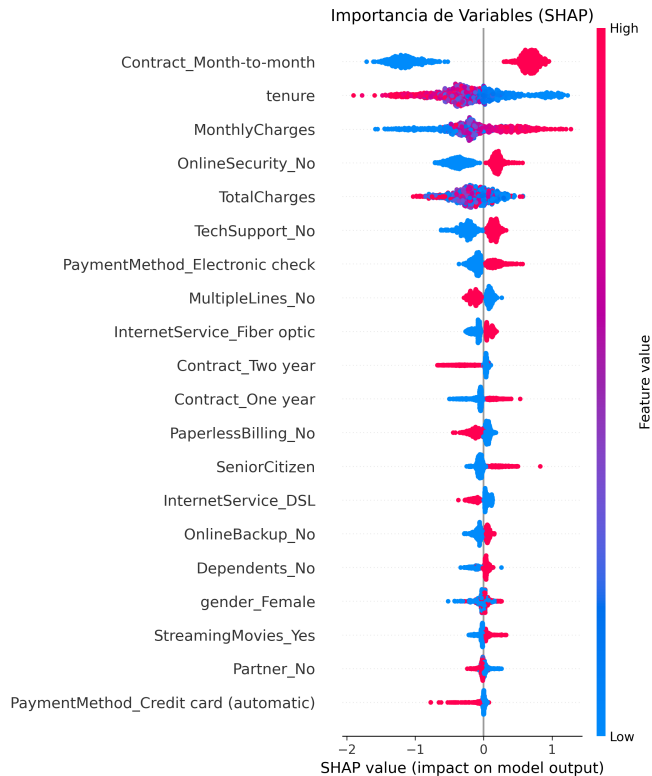


Figura 4: Importancia global de características según SHAP para el modelo óptimo.

5.4. Ejemplo de explicación generada por el LLM

Para demostrar el comportamiento del sistema en un caso real, se seleccionó un cliente del conjunto de datos de prueba (índice 1000) mediante la interfaz interactiva. La predicción del modelo, los factores SHAP y el reporte explicativo generado por el LLM local se exponen de forma gráfica en la Figura 5.

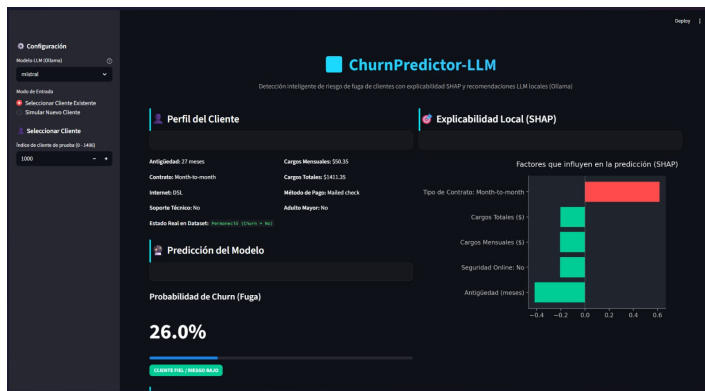
El cliente analizado posee una antigüedad de 27 meses, contrato mensual (*Month-to-month*), cargos mensuales de \$50.35, cargos totales de \$1411.35 y método de pago por cheque físico (*Mailed check*). Para este perfil, el modelo XGBoost estimó un riesgo de fuga de **26.0 %**, lo cual lo cataloga como un cliente fiel o de bajo riesgo. El análisis local SHAP en la Figura 5(a) revela que, mientras el contrato mensual actúa incrementando el riesgo (barra roja de impacto positivo), factores como la antigüedad y los cargos mensuales actúan reduciendo de manera significativa el riesgo de fuga (barras verdes de impacto negativo).

Posteriormente, estos factores SHAP y la probabilidad estimada se enviaron a Ollama con el modelo *mistral* para generar recomendaciones automáticas, presentadas en el panel interactivo (Figura 5(b)). El texto obtenido se transcribe íntegramente a continuación:

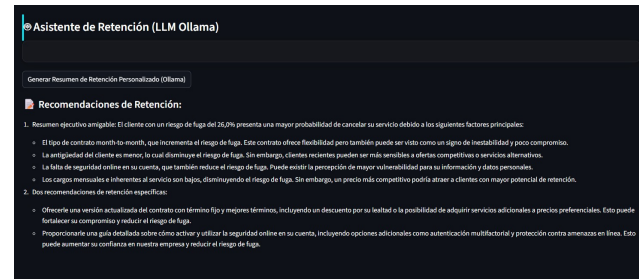
Reporte Generado por Ollama (Mistral)

Recomendaciones de Retención:

- Resumen ejecutivo amigable:** El cliente con un riesgo de fuga del 26 % presenta una mayor probabilidad de cancelar su servicio debido a los siguientes factores principales:
 - El tipo de contrato *month-to-month*, que incrementa el riesgo de fuga. Este contrato ofrece flexibilidad pero también puede ser visto como un signo de inestabilidad y poco compromiso.
 - La antigüedad del cliente es menor, lo cual disminuye el riesgo de fuga. Sin embargo, clientes recientes pueden ser más sensibles a ofertas competitivas o servicios alternativos.
 - La falta de seguridad online en su cuenta, que también reduce el riesgo de fuga. Puede existir la percepción de mayor vulnerabilidad para su información y datos personales.
 - Los cargos mensuales e inherentes al servicio son bajos, disminuyendo el riesgo de fuga. Sin embargo, un precio más competitivo podría atraer a clientes con mayor potencial de retención.
- Dos recomendaciones de retención específicas:**
 - Ofrecerle una versión actualizada del contrato con término fijo y mejores términos, incluyendo un descuento por su lealtad o la posibilidad de adquirir servicios adicionales a precios preferenciales. Esto puede fortalecer su compromiso y reducir el riesgo de fuga.
 - Proporcionarle una guía detallada sobre cómo activar y utilizar la seguridad online en su cuenta, incluyendo opciones adicionales como autenticación multifactorial y protección contra amenazas en línea. Esto puede aumentar su confianza en nuestra empresa y reducir el riesgo de fuga.



(a) Predicción del modelo y factores explicativos SHAP.



(b) Recomendaciones de retención generadas localmente por Ollama.

Figura 5: Capturas de pantalla del panel interactivo de Streamlit para el cliente de prueba 1000.

5.5. Resumen de resultados

- **Mejor modelo:** XGBoost con GridSearch (AUC=0.84, F1=0.63).
- **Interpretabilidad:** SHAP indica que `tenure`, `monthlyCharges` y `contract` son los atributos más influyentes.
- **Valor añadido del LLM:** generación automática de explicaciones comprensibles que pueden integrarse en sistemas de atención al cliente o reportes ejecutivos.

Con estos resultados el pipeline completo (pre-procesamiento, entrenamiento, evaluación, explicabilidad) está listo para ser desplegado y para que los analistas de negocio obtengan insights accionables sobre el churn de los clientes.

6. Discusión

6.1. Interpretación de resultados

El objetivo principal planteado en la introducción se cumplió con éxito. El clasificador XGBoost optimizado mediante búsqueda de cuadrícula alcanzó un **AUC-ROC = 0.84** y un **F1-Score = 0.63** en el conjunto de prueba independiente, superando el umbral esperado de 0,80 y mejorando al baseline de Regresión Logística (AUC = 0,83, F1 = 0,61). Cabe destacar que el XGBoost sin ajuste de hiperparámetros obtuvo un desempeño inferior en el F1-Score (0,56), lo cual demuestra que el ajuste de la tasa de aprendizaje ($\eta = 0,05$), la profundidad máxima de los árboles (5) y la compensación del desbalance mediante `scale_pos_weight = 3` fueron factores determinantes para evitar el sobreajuste y mejorar la capacidad de generalización del modelo.

En el plano empresarial, el modelo destaca por alcanzar un **Recall = 0.79**. Esto indica que el sistema es capaz de detectar aproximadamente al 79 % de los clientes con intenciones reales de abandonar la compañía (verdaderos positivos), reduciendo al mínimo los falsos negativos. La

precisión resultante (0,52) implica la existencia de falsas alarmas; sin embargo, en un escenario de retención de clientes, el costo de implementar una campaña preventiva sobre un cliente que no iba a abandonar es significativamente menor que el costo de perder a un usuario activo.

El análisis de explicabilidad con SHAP validó que el tipo de contrato mensual (`Contract_Month-to-month`), la antigüedad (`tenure`) y los cargos mensuales (`MonthlyCharges`) son las variables con mayor relevancia global. Finalmente, la integración con el LLM local Ollama resolvió con éxito la brecha entre las puntuaciones matemáticas abstractas de SHAP y la interpretación intuitiva que requiere el personal no técnico de mercadeo y servicio al cliente, proveyendo un flujo explicativo y recomendaciones coherentes en español.

6.2. Limitaciones

A pesar del buen rendimiento general, el proyecto presenta ciertas limitaciones importantes:

- **Naturaleza estática de los datos:** El conjunto de datos de IBM representa una foto instantánea en el tiempo. En entornos reales, el comportamiento de los clientes es dinámico y secuencial (p.ej., caídas repentinas en el consumo de datos), lo cual no se captura con variables tabulares estáticas.
- **Desbalance residual:** A pesar de ponderar las clases, la relación de desbalance de 3:1 limita la precisión del modelo, generando un número considerable de falsos positivos.
- **Restricciones computacionales locales:** La ejecución del LLM local (`mistral`) mediante Ollama está limitada por la memoria RAM y capacidad de procesamiento de la CPU/GPU del equipo local. Esto causa latencias en la generación de textos (varios segundos por consulta) y limita la posibilidad de usar modelos de mayor tamaño (p.ej., de 70B parámetros) que proporcionarían análisis aún más sofisticados.

6.3. Trabajo futuro

Como líneas de desarrollo futuro para el sistema propuesto, se identifican las siguientes opciones:

- **Modelado temporal e histórico:** Implementar modelos de aprendizaje profundo para series de tiempo (como LSTM o Transformers) o técnicas de supervivencia para predecir no solo si el cliente abandonará, sino el momento exacto en el que lo hará.
- **Integración RAG (Retrieval-Augmented Generation):** Conectar el LLM local a una base de conocimientos con los registros históricos de campañas previas y guías internas de la empresa, de modo que las recomendaciones de Ollama se basen en políticas y promociones reales vigentes.
- **Despliegue distribuido en la nube:** Migrar la inferencia del modelo ML y del LLM a una arquitectura en la nube (p.ej., usando APIs de modelos de lenguaje más robustos y hardware dedicado) para eliminar la latencia local y permitir su uso simultáneo por múltiples agentes de servicio en tiempo real.

7. Conclusiones

En este trabajo se diseñó e implementó un flujo completo para la predicción e interpretación de la fuga de clientes, integrando aprendizaje automático y modelos de lenguaje de gran tamaño. A partir de los resultados obtenidos, se desprenden las siguientes conclusiones principales:

- Se respondió afirmativamente a la pregunta de investigación: el modelo XGBoost optimizado logró predecir el churn con un **AUC-ROC de 0.84** (superando el umbral propuesto de 0,80) y un **F1-Score de 0.63**, superando al baseline de regresión logística por 0,01 y 0,02 puntos absolutos en AUC y F1, respectivamente.
- La etapa de sintonización de hiperparámetros mediante GridSearchCV demostró ser esencial para la generalización del modelo. Mientras que la configuración por defecto de XGBoost tendió al sobreajuste, la limitación de profundidad y el uso de regularización permitieron equilibrar las métricas de entrenamiento y validación.
- La explicabilidad local a través de SHAP aportó transparencia matemática al proceso de inferencia de la caja negra, identificando de manera consistente que la flexibilidad de los contratos mensuales y la baja antigüedad son los mayores detonantes del churn de clientes.
- El uso de un LLM local (mistral) sirvió de puente comunicativo, automatizando la traducción de métricas y contribuciones de variables complejas en reportes ejecutivos con sugerencias de acción específicas. Esto valida la viabilidad de acoplar modelos predictivos robustos con tecnologías de IA generativa para generar valor real en la operación del negocio.

Contribuciones del equipo

Integrante	Contribución principal
Mariana Carrasquilla	Data Science, EDA, ML, documentación
Sofia Gallo	LLM, Backend, Reproducibilidad, documentación

Distribución de responsabilidades por integrante.

Declaración de Uso de Inteligencia Artificial

En cumplimiento de las directrices académicas, se declara el uso del modelo de lenguaje **Google Gemini** (a través del entorno de desarrollo cooperativo) como herramienta de soporte, asistencia y co-creación de código. La participación de la inteligencia artificial abarcó:

- Generación de código y asistencia en el desarrollo de scripts en Python (incluyendo la creación del script independiente de visualización de curvas de aprendizaje y el ajuste estructurado del modelo XGBoost).
- Soporte en la optimización y depuración de código fuente existente.
- Asistencia técnica en la estructuración, formateo sintáctico e integración de figuras en el reporte en LaTeX (proyecto.tex).
- Revisión de la consistencia numérica entre las métricas reales del modelo y las tablas de resultados del documento.

Toda la lógica de diseño experimental, la toma de decisiones metodológicas, la interpretación de resultados y la redacción final de este documento científico fueron de autoría exclusiva del equipo de estudiantes.

Repositorio y Demo

- **GitHub:** <https://github.com/mcarrasqub/ChurnPredictor-LLM>
- **Video demo:** <https://youtu.be/q-tMOMN85VM>
- **Instalación:** Documentada detalladamente en el archivo README.md del repositorio.

Referencias

[1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems (NeurIPS), 30. <https://arxiv.org/abs/1706.03762>

[2] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP*

- Tasks. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
- [3] Chase, H. (2022). *LangChain: Building applications with LLMs through composability*. Repositorio GitHub. <https://github.com/langchain-ai/langchain>
- [4] IBM Cognitive Class. (2020). *Telco Customer Churn Dataset*. Kaggle. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- [5] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD, 785–794. <https://arxiv.org/abs/1603.02754>
- [6] Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS), 30. <https://arxiv.org/abs/1705.07874>
- [7] Google DeepMind. (2026). *Gemini: Advanced Agentic Coding Assistant*. Google AI. <https://deepmind.google/technologies/gemini/>